

CHATBOT DEVELOPMENT PROJECT REPORT

Bemo — Smart AI Assistant powered by Google Gemini 2.5 Flash

Project 2: Chatbot Development Challenge

Group : AI 2



1. Project Overview

Bemo is a fully-featured, production-grade AI chatbot assistant built on top of Google's Gemini 2.5 Flash model. Developed as part of Project 2: Chatbot Development Challenge, Bemo goes far beyond a simple question-answering bot — it integrates real-time web search, weather data, mathematical computation, file summarization, image analysis, and full voice I/O into a cohesive, dark-themed desktop application.

1.1 Project Objectives

- Build a functional AI chatbot that responds correctly and naturally to diverse user inputs
- Handle unexpected or out-of-scope inputs gracefully without crashing or giving nonsensical replies
- Support multi-turn conversations with context retention across the full session
- Avoid simple if/else-only logic — use an LLM-powered agent pipeline instead
- Integrate at least one advanced feature beyond basic chat (Bemo implements five: web search, weather, calculator, file summary, image analysis, and voice I/O)
- Deliver clean, modular, well-documented code organized in logical sections

1.2 Feature Summary Table

Feature	Technology	Capability	Input Mode
 LLM Core	Gemini 2.5 Flash	Natural language understanding & generation	Text / Voice / Image
 Web Search	DuckDuckGo (ddgs)	Real-time web results (top 3)	Text query
 Weather	wtrr.in API	Current weather for any city	City name
 Calculator	SymPy + popup UI	Symbolic & numeric math	Expression string
 Date/Time	Python datetime	Current date and time	Natural language trigger
 File Summary	Gemini + file I/O	Summarizes text files up to 8000 chars	File upload
 Memory	In-memory list	Full conversation history in prompt	Automatic
 Image Vision	Gemini Vision	Describe & analyze any uploaded image	Image upload
 Live Camera	OpenCV + Gemini	Capture & analyze webcam frame	Webcam button
 TTS (Voice)	pyttsx3 (offline)	Bemo speaks her replies aloud	Automatic / toggle

Feature	Technology	Capability	Input Mode
🗣️ STT (Voice)	SpeechRecognition	Speak instead of typing	Microphone button

2. System Architecture

Bemo is built as a layered agent pipeline. Each layer has a single responsibility, making the system modular, testable, and easy to extend.

2.1 High-Level Pipeline

#	Layer	Responsibility	Key Component
1	Input Layer	Accept text, voice, file, or image input from user	Tkinter GUI + STT + filedialog + OpenCV
2	Tool Router	Classify intent and select the right tool	fast_decide() + decide_tool() + Gemini fallback
3	Tool Execution	Run the selected tool and collect its result	TOOL_FN dispatch dict + individual tool functions
4	Response Generator	Build a rich prompt and call Gemini for a natural reply	generate_response() + PERSONALITY system prompt
5	Memory Layer	Save turn to history; inject history into next prompt	save_memory() + get_memory()
6	Output Layer	Display reply bubble in GUI and optionally speak it	add_bubble() + speak() TTS

2.2 Model Wrapper: `_ModelWrapper`

The raw `GenerativeModel` is wrapped in a custom `_ModelWrapper` class that adds automatic retry logic for 429 rate-limit errors. Key behaviors:

- Retries up to 5 times on quota errors with exponential back-off (starting at 20s, capped at 120s)
- Parses the suggested retry delay from the error message and adds a 2-second buffer
- Shows a live countdown in the status bar while waiting, keeping the UI informative
- Re-raises immediately for all non-quota errors — no silent failures
- All other API calls throughout the app go through this wrapper, ensuring consistent resilience

3. Tool Routing System

The tool routing system is a two-stage hybrid classifier that decides which tool to invoke for a given user message. This avoids simple if/else-only logic by combining rule-based efficiency with LLM-powered flexibility.

3.1 Stage 1: Rule-Based Pre-Filter (fast_decide)

Before making any API call, fast_decide applies keyword and heuristic rules to catch obvious cases instantly:

Target Tool	Detection Rule	Example Triggers
WEATHER	Word set intersection with {weather, الطقس, جو, حرارة, درجة}	"weather in Cairo", "طقس القاهرة"
DATETIME	Word set intersection with {time, date, اليوم, الساعة, وقت, تاريخ, today}	"what time is it", "النهاره إيه"
CALCULATE	Math char density > 60% of input length	"(3+4)*2", "15/3"
(unknown)	None of above — falls through to Stage 2	General questions, search queries

3.2 Stage 2: LLM Fallback (decide_tool)

If fast_decide returns None, a minimal Gemini prompt is sent asking for a single keyword from the set {SEARCH, CALCULATE, WEATHER, DATETIME, NONE}. The response is parsed with substring matching to extract the final decision. This ensures correct routing for ambiguous, complex, or mixed-language queries without requiring manual rule expansion.

3.3 Tool Dispatch

The TOOL_FN dictionary maps each keyword to its function:

Keyword	Function	Returns	External Call?
SEARCH	web_search(query)	Top 3 DuckDuckGo result bodies	Yes (DDGS)
CALCULATE	calculator(query)	SymPy evaluated result string	No (local)
WEATHER	get_weather(city)	wtr.in formatted weather string	Yes (wtr.in)
DATETIME	get_datetime()	Formatted date+time string	No (local)

Keyword	Function	Returns	External Call?
NONE	(no tool called)	None — Gemini answers from knowledge	No

4. Individual Tools

4.1 Web Search

Uses the DuckDuckGo Search library (ddgs) to retrieve the top 3 organic search results for any query. The body text of each result is extracted and joined into a single string that is injected into the Gemini prompt as a "Tool Result" block. This gives Bemo access to current events and factual data beyond her training cutoff without requiring a paid API key.

4.2 Calculator

Powered by SymPy's sympify() function, the calculator tool handles both numeric evaluation and symbolic mathematics. It accepts any valid mathematical expression as a string and returns the simplified result. Errors are caught gracefully with an "Error in calculation" fallback. Additionally, a standalone popup calculator window provides a user-friendly button interface for manual computation.

Button Group	Buttons	Color Theme	Function
Clear/Delete	C, ✕	Red (CALC_CLR)	Clear all / backspace
Operators	/, *, -, +, %	Pink-Red (CALC_OP)	Arithmetic operators
Equals	=	Purple (CALC_EQ)	Evaluate with eval()
Digits	0-9, 00, .	Dark Blue (CALC_NUM)	Numeric input

4.3 Weather

Calls the free wttr.in API with a city name and format=3, which returns a concise one-line weather string (e.g., "Cairo: 🌤️ +32°C"). The request is wrapped in a try/except to return "Weather unavailable" gracefully if the service is down or the city name is invalid.

4.4 Date & Time

Uses Python's datetime.now() to return the current date and time formatted as a friendly, emoji-decorated string: "📅 Friday, 08 May 2026 | 🕒 09:30 PM". No external calls are needed — this is a fully local, zero-latency tool.

4.5 File Summarization

Accepts any text-based file (txt, py, md, csv, json, html, xml) via a file dialog. The file is read with UTF-8 encoding (errors ignored), and the first 8,000 characters are sent to Gemini with a summarization prompt under Bemo's persona. The summary is displayed as a chat bubble and saved to conversation memory, making it referenceable in follow-up questions.

4.6 Image Analysis (Vision)

Leverages Gemini 2.5 Flash's multimodal vision capabilities to analyze uploaded images. The PIL.Image object is passed directly to the Gemini API alongside a contextual prompt. The system supports:

- Image Upload — user selects any JPG, PNG, BMP, GIF, or WEBP file via file dialog
- Live Camera Capture — OpenCV opens a live webcam preview in a Toplevel window; user clicks 📷 Capture to snapshot a frame
- Inline Thumbnails — a resized 260x200 preview is embedded directly in the chat bubble
- Contextual Questions — the user's current text input is used as the question about the image

5. Conversation Memory & Personality

5.1 Memory Architecture

Bemo maintains full conversation history as a list of {"user": ..., "bot": ...} turn dictionaries. The entire history is serialized and injected into every Gemini prompt as a "Full conversation so far" section. This gives Bemo true multi-turn context awareness — she can reference anything said earlier in the conversation naturally, without being prompted to do so.

Function	Description
save_memory(user, bot)	Appends a new turn dict to the memory list after each response
get_memory()	Serializes the full history as "User: ... Bemo: ..." lines for prompt injection
memory_count()	Returns the number of stored turns — displayed live in the status bar
clear_chat()	Clears memory and resets the chat window via the Clear button

Image and file summaries are also saved to memory (e.g., "[Image: photo.png]" → summary), making them part of the context for follow-up text questions.

5.2 Bemo's Personality System Prompt

A rich PERSONALITY constant defines Bemo's communication style and is prepended to every `generate_response()` call. Key behavioral guidelines:

- Warm and conversational: responds like a smart friend, not a formal corporate assistant
- Concise and clear: no unnecessary filler or padding in answers
- Dry humor: light wit when contextually appropriate, never forced
- Multilingual: automatically mirrors the user's language (Arabic, English, or mixed)
- Adaptive explanation: re-explains in simpler terms when the user seems confused, without condescension
- Natural context use: references earlier conversation points organically when relevant
- Authentic voice: never uses "As an AI..." disclaimers — just responds helpfully and directly

6. Voice Features

6.1 Text-to-Speech (TTS) — Bemo Speaks

Implemented with `pyttsx3` (fully offline), the TTS engine runs in a background thread to avoid blocking the GUI. Key implementation details:

- Speaking rate set to 165 wpm, volume at 100% for clear output
- A `threading.Lock` ensures only one speech thread runs at a time — no overlapping audio
- Markdown symbols (`*`, `#`, ```) are stripped from text before speaking to avoid robotic filler words
- A  ON /  OFF toggle button in the header lets the user enable or disable voice at any time
- TTS is called automatically after every Bemo reply (when enabled) with no user action required

6.2 Speech-to-Text (STT) — Speak Instead of Type

Powered by the `SpeechRecognition` library with Google's speech recognition backend, the STT feature is triggered by the  microphone button:

- Ambient noise adjustment (0.4s calibration) before listening begins
- Timeout of 8 seconds for speech start; 15-second phrase limit
- Recognized text is inserted into the input field (not auto-sent) — user can review before sending
- Three distinct error states: `WaitTimeoutError` (no speech), `UnknownValueError` (unclear speech), `RequestError` (service failure)
- All listening runs in a background daemon thread — the GUI remains fully responsive
- Live status bar updates:  Listening... →  Recognising... → result or error

7. Graphical User Interface

Bemo's GUI is built with Tkinter and styled with a dark Catppuccin Mocha-inspired color palette, closely resembling WhatsApp's dark mode for a familiar and polished experience.

7.1 Color Palette

Variable	Hex Code	Role	Inspired By
BG	#1e1e2e	Window background	Catppuccin Mocha Base
CHAT_BG	#181825	Chat area background	Catppuccin Mocha Crust
INPUT_BG	#313244	Input field / buttons	Catppuccin Surface0
BTN_BG	#89b4fa	Primary button	Catppuccin Blue
USER_BUBBLE	#005c4b	User chat bubbles	WhatsApp dark green
BEMO_BUBBLE	#202c33	Bemo chat bubbles	WhatsApp dark grey
BEMO_NAME	#53bdeb	Bemo's name label	WhatsApp light blue

7.2 Layout Components

Header Bar: Application title, "Powered by Gemini 2.5 Flash" subtitle, 📊 Calc button, 🔊 TTS toggle, and Clear button

Chat Area: Scrollable Canvas + inner Frame; auto-scrolls to bottom on new messages; mouse wheel support

Input Row: 📎 File upload, 🎤 Microphone, 📷 Camera, text Entry field (Enter key = send), and Send ➤ button

Status Bar: Live status messages + memory turn count (e.g., "Ready | 💬 3 message(s) in memory")

7.3 Chat Bubbles

Chat bubbles are rendered as Canvas widgets with smooth rounded rectangles (corner radius 18px). Each bubble:

- Uses manual text wrapping with font measurement to calculate exact canvas dimensions
- Supports right-click context menu with 📄 Copy option
- Supports double-click to copy text to clipboard
- Shows a flash animation (background color change for 180ms) when text is copied
- User messages align right (dark green); Bemo messages align left (dark grey)
- Wide margins prevent bubbles from spanning the full chat width

7.4 Threading Model

All blocking operations run in background daemon threads to keep the GUI responsive:

- `send_message()`: `agent()` call runs in `threading.Thread`; UI updated via `root.after(0, ...)`
- `upload_file()`: file reading and `summarize_file()` call are threaded
- Image analysis: `analyze_image()` call is threaded
- TTS: each `speak()` call runs in its own daemon thread with a Lock
- STT: `listen_once()` runs entirely in a background thread with callback-based result delivery

8. Meeting the Project Requirements

Requirement	How Bemo Meets It
Responds correctly to user inputs	Gemini 2.5 Flash with tool augmentation ensures accurate, context-aware answers for text, images, math, weather, and search queries
Handles unexpected/wrong inputs gracefully	All tool functions have try/except blocks with meaningful fallback messages; the NONE route lets Gemini handle anything not covered by tools
Multi-turn conversations with context	Full conversation history injected into every prompt via <code>get_memory()</code> ; Bemo naturally references earlier turns
Not just static if/else logic	Two-stage LLM-backed routing, dynamic Gemini responses, and tool augmentation entirely replace static response trees
Natural and smooth conversation flow	PERSONALITY system prompt enforces concise, warm, contextual replies; no robotic disclaimers
Greeting and fallback responses	"Hello! Ask me anything 🗣️" greeting on launch; NONE route provides graceful fallback for any unknown query
Memory of previous interactions	Full session history retained in memory list; count displayed live in status bar
External API integration	DuckDuckGo web search, wttr.in weather, Gemini AI, and Google Speech Recognition
Clean and well-structured code	10 numbered sections in notebook; each feature in its own function/class; clear naming conventions
Proper use of functions and classes	<code>_ModelWrapper</code> class; dedicated functions for every tool, routing stage, GUI component, and voice feature
Modular and readable organization	Notebook divided into Install → API → Model → Tools → Router → Personality → Agent → GUI → Voice → Camera

Requirement	How Bemo Meets It
Comments and documentation	Every section has a markdown header explaining purpose; inline comments on non-obvious logic; docstrings on key functions

9. Code Quality & Implementation

9.1 Modular Design

The codebase follows strict separation of concerns across 10 notebook sections:

- Section 1 (Install): All pip dependencies in one place — easy to update or swap
- Section 2 (API Key): Secure key injection via getpass — never hardcoded
- Section 3 (Model): `_ModelWrapper` encapsulates all resilience logic
- Section 4 (Tools): Each tool is a standalone function — independently testable
- Section 5 (Router): Routing logic isolated from tools and response generation
- Section 6 (Personality & Response): Prompt engineering separated from agent dispatch
- Section 7 (Agent): Thin orchestration layer — glues routing, tools, response, memory
- Section 8 (GUI): All UI code isolated; clean factory functions for reusable components
- Section 9 (Voice): TTS and STT each in separate functions with clear callback APIs
- Section 10 (Camera): Vision and capture logic in dedicated functions

9.2 Error Handling Strategy

Every tool function and I/O operation is wrapped in try/except:

- `web_search`: DDGS errors return an empty list gracefully
- `calculator`: sympify exceptions return "Error in calculation"
- `get_weather`: requests exceptions return "Weather unavailable"
- `summarize_file`: file I/O errors return "Could not read file: {e}"
- `analyze_image`: API errors return "Sorry, I couldn't analyse the image: {e}"
- `capture_from_camera`: camera open failure calls `on_error` callback
- `listen_once`: three distinct STT error types with specific, user-friendly messages
- `send_message`: top-level try/except catches any `agent()` failure with a friendly error bubble

9.3 Dependencies

Package	Version/Source	Purpose	Local/Remote
google-generativeai	PyPI	Gemini LLM + Vision API	Remote

Package	Version/Source	Purpose	Local/Remote
ddgs	PyPI (duckduckgo-search)	Web search without API key	Remote
sympy	PyPI	Symbolic math evaluation	Local
requests	PyPI	Weather API HTTP call	Remote
pyttsx3	PyPI	Offline TTS engine	Local
SpeechRecognition	PyPI	Microphone → text (Google STT)	Remote (Google API)
opencv-python	PyPI	Webcam capture	Local
Pillow	PyPI	Image loading + thumbnails	Local
tkinter	Python stdlib	GUI framework	Local

10. Conclusion

Bemo represents a complete, production-quality chatbot implementation that goes significantly beyond the minimum project requirements. By combining a powerful LLM backbone with intelligent tool routing, rich multimodal inputs, full voice I/O, and a polished desktop GUI, Bemo delivers a genuinely useful and technically sophisticated AI assistant.

10.1 Technical Highlights

- Hybrid tool router (rules + LLM) avoids both the brittleness of pure if/else and the latency of pure LLM routing for every message
- Full prompt engineering: PERSONALITY system prompt + history injection + tool result injection = rich, context-aware replies
- Non-blocking threading throughout ensures the GUI never freezes regardless of API latency
- Exponential back-off retry logic makes the app resilient to Gemini rate limits in real-world use
- Bilingual support (Arabic + English) baked into both the router keywords and Bemo's personality prompt
- Multimodal: handles text, uploaded files, images, live camera, and voice — all in one interface

10.2 Additional Features Beyond Requirements

Extra Feature	Value Added
 Image Vision (upload + camera)	Makes Bemo a true multimodal assistant
 TTS +  STT	Hands-free interaction; accessibility benefit
 Pop-up Calculator	Standalone utility embedded in the app
 File Summarization	Knowledge extraction from local documents
 Rate-Limit Retry with Countdown	Production resilience; transparent UX feedback
WhatsApp-style Bubble UI	Familiar, polished, professional appearance
Copy on double-click + context menu	Practical UX detail
Bilingual router (Arabic + English)	Broader user accessibility

Link on GitHub :

https://github.com/ahmedmohamed1807/Bemo_Chat_Bot/tree/main

